CSC591 Homework 1
Student: Tzu Tang(Leslie) Liu
Unity ID: tliu33

Github: https://github.com/LeslieLiu11/GenAI-for-Systems-Gym

(collect task scripts in homework1/scripts, logs file in homework1/logs, and tensorboard file in homework1/models/LSTM/experiment)

Section I

Deliverables [Section I: Question 1-6] :

Conceptual Questions:

The previous subsections give background on cache replacement and the paper in discussion. At this point, you should have thoroughly read the paper **An Imitation Learning Approach for Cache Replacement**. The following questions are supposed to demonstrate your understanding of the paper.

Question I) The authors mention two approaches to improve cache performance (hit rate). Which approach does the paper focus on?

The paper discusses two main methods for improving cache performance:

(1)Prefetching: This technique involves loading data into the cache before it's needed, which helps prevent cache misses.

(2)Cache Replacement: This method focuses on deciding which cache line to remove when new data needs to be inserted into a full cache.

The paper specifically concentrates on cache replacement. It reformulates the problem as an imitation learning task. The authors introduce a replacement policy called **PARROT**, which is designed to approximate Belady's optimal eviction policy. Belady's policy is considered theoretically optimal because it has knowledge of future memory accesses and serves as an oracle during the training process. Once trained, PARROT relies solely on past cache access data to make its eviction decisions, allowing it to perform near-optimally even with complex access patterns.

Question II) What are the previous state-of-the-art approaches mentioned in the paper?

Based on the paper, there are some previous state-of-the-art approaches:

Hawkeye (Jain & Lin, 2016) uses a binary classifier to decide whether a cache line is likely to be reused soon (deemed "cache-friendly") or not (deemed "cache-averse"). Evicts the cache-averse lines first, and then relies on a conventional heuristic to choose which cache-friendly line to evict if needed.

Glider (Shi et al., 2019) also adopts a learning-based method to predict cache line reuse, aiming to better handle diverse and complex access patterns. He use a different architecture and training objective compared to Hawkeye, yet still relies on a combination of learned predictions and traditional heuristics for making final eviction decisions.

Question III) Explain the cache hierarchy used for experimentation by the authors. This means you have to give the size of L1/L2/L3 caches including the number of sets and ways.

There are three different level of cache used for experimentation by the authors:

L1 Cache:

- Size: 32 KB
- Organization: 4-way set-associative
- Details: With 64-byte cache lines, there are 512 lines total, organized into 128 sets.

L2 Cache:

- Size: 256 KB
- Organization: 8-way set-associative
- Details: Using 64-byte lines, it contains 4096 lines, arranged in 512 sets.

L3 Cache:

- Size: 2 MB
- Organization: 16-way set-associative
- Details: With 64-byte lines, there are 32,768 lines, divided into 2048 sets. This high associativity helps reduce conflicts and improves hit rates for complex memory access patterns.

Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

Question IV) What is the normalized cache hit rate? Why have the authors used it as a metric?

The normalized cache hit rate is defined as (r – r_LRU) / (r_opt – r_LRU)

where:
r is the hit rate of the policy being evaluated,
r_LRU is the hit rate using the traditional LRU policy
r_opt is the hit rate achieved by Belady's optimal policy.

This metric shows what fraction of the performance gap between LRU and the optimal policy is closed by a given approach. The authors use it to compare different cache replacement methods on a common scale, making it easier to see how close each method comes to the theoretical optimum regardless of the absolute hit rate differences across

Question V) Interpret Figure 2 from the paper. Explain what you understand by looking at the figure and the caption.

Figure 2 plots the normalized cache hit rate of a variant of Belady's algorithm that uses a limited future window. The y-axis is scaled so that 0 represents the hit rate of LRU and 1 represents the full performance of Belady's optimal policy. In essence, the graph shows how much of the optimal performance is achieved as more future cache accesses are considered. The key takeaway is that to reach about 80% of the optimal hit rate, the algorithm needs to accurately predict reuse distances up to roughly 2600 future accesses, highlighting the challenge of making eviction decisions without complete future information.

Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

Question VI) The following table shows the number of misses encountered while evaluating/testing the Parrot model in the second column. The last column estimates the total number of cache misses, i.e., the speculated number of cache misses for the complete trace and all cache sets. What formula might be used to calculate the values in the last column? Explain your answer.

A	B	C
Trace	# of misses (test)	Total # of misses
lbm_564B	95424	30535680
astar_313B	94178	30136960
milc_409B	91172	29175040

The formula might be used as:
M_obs × (L_total / L_sample) × (S_total / S_sample)

where:
M_obs is the number of misses observed during evaluation,
L_sample is the length of the trace used (with L_total being the complete trace length)
S_sample is the number of cache sets sampled (with S_total being the total number of sets in the cache)

This formula extrapolates the measured misses by accounting for both the fraction of the trace evaluated and the fraction of cache sets observed, thereby providing an estimate of the total misses over the complete trace and all cache sets.

Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

Section II

Deliverables [Section II: Task 1-2]:

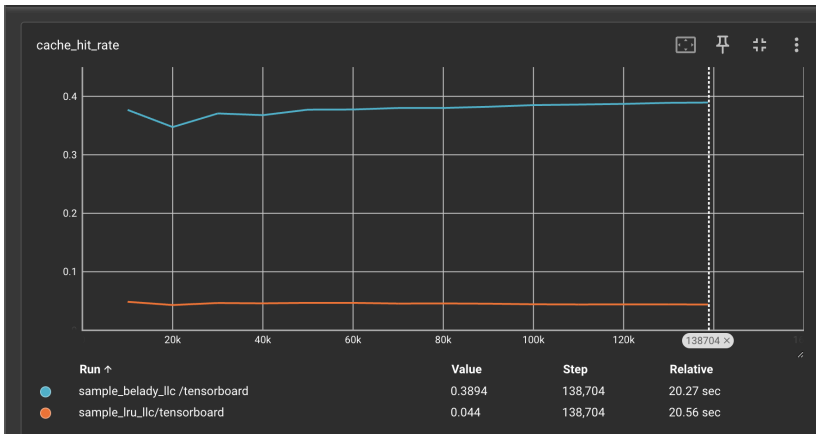
Task I

Follow the steps shown in the GitHub repository and find the raw cache hit rate for the **LRU** replacement policy. Use the astar_313B_test.csv trace file.

Task II

Follow the steps shown in the GitHub repository and find the raw cache hit rate for the **Belady** replacement policy. Use the astar_313B_test.csv trace file.

Task1 and Task2 Tensorboard result:



By the result from tensorboard, we get:
RLU:
Raw hit rate: 0.044
Belady::
Raw hit rate: 0.3894

Belady (optimal) replacement policy achieves a significantly higher raw cache hit rate (0.3894) compared to the LRU replacement policy (0.044). This large gap is expected because Belady has “oracle” knowledge of future accesses, enabling it to evict the block that will not be used for the longest time. LRU, on the other hand, makes eviction decisions based only on past usage, without knowing future accesses.

Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

Section III

You will run all the models in this homework only on one trace file. You may find the astar_313B trace files (train, valid & test) in the "traces" directory. In this section you will perform model space exploration for a Multi Layer Perceptron model. You may find the code for this in "models/MLP". You shall make changes to the model configurations using command arguments or by manually editing parts of the code. Some guidelines are provided below for help.

Note: For all the tasks below and in the subsequent sections, evaluate your trained models at checkpoint 20,000 (20000.ckpt) to ensure consistency across results.

Default model configuration: (as is in default.json)

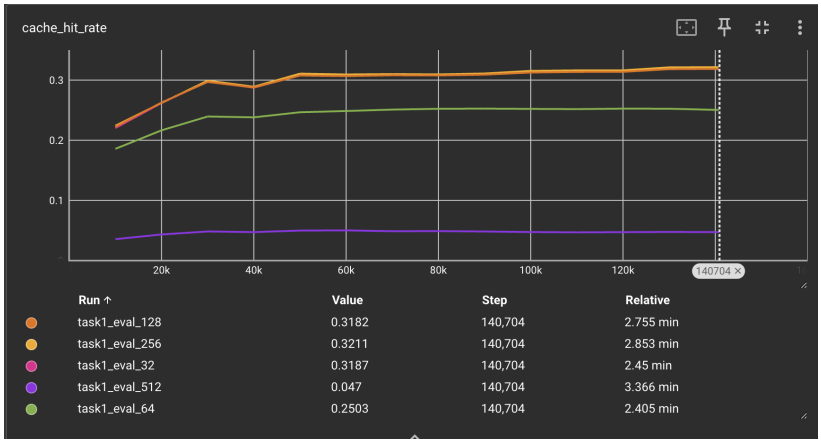
Number of neurons - 128
Number of layers - 2 (as in the default code provided)
Activation Functions - ReLU
Batch size - 32

Deliverables [Section III: Task 1-6]:

Task I

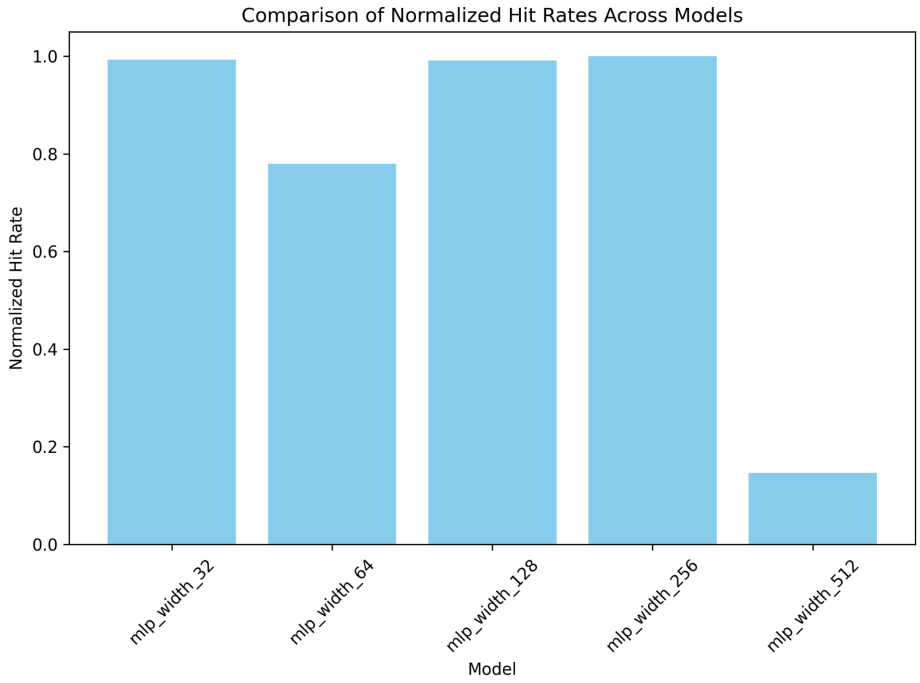
Vary the size/width (number of neurons in a layer) of the single-layer perceptron model and plot the normalized cache hit rate (y-axis) v/s the size of perceptron (x-axis). Plot the following number of neurons - 32,64,128,256,512. You may change this by changing the model bindings - "lstm_hidden_size" to the desired number of neurons.

Raw hit rate for neurons size 32,64,128,256,512:
We found that size 128 and 256 have better performance, but neurons size 512 have a lower performance in that case, so the result suggests around 128–256 neurons for this particular workload and model architecture. While increasing model capacity can help capture more complex patterns, it can also lead to diminishing returns and potentially overfitting.



Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

Plot for normalized hit rate:

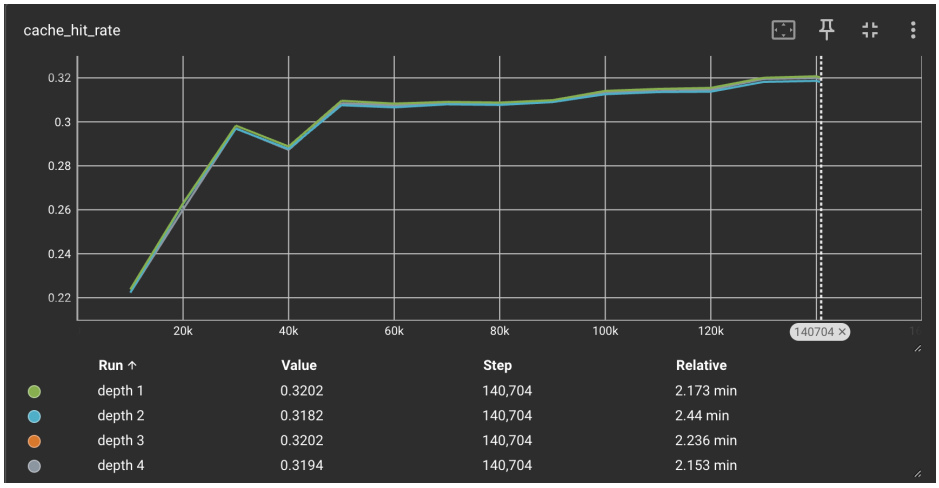


Task II

Vary the depth (number of layers) of the multi-layer perceptron model and plot the normalized cache hit rate (y-axis) v/s the number of layers (x-axis). Keep the size default at 128 neurons/layer. Plot the following number of layers - 1,2,3,4. Manually add code to add more layers. Use the default activation function - ReLU.

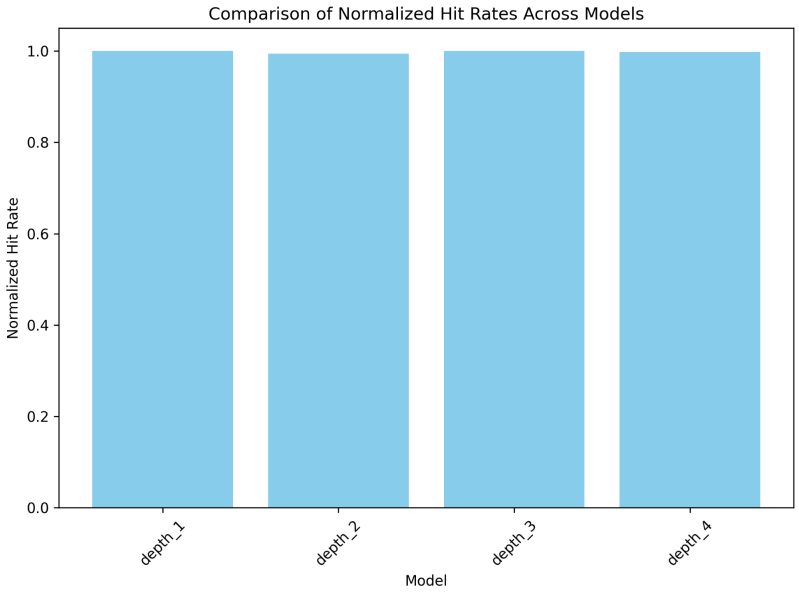
Raw hit rate from 1 layer to 4 layers:

These results suggest that adding more layers does not necessarily translate into better performance for this task and dataset. The single-layer model performs as well as deeper models, which could indicate that the dataset or training setup does not require extra model capacity.



Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

Plot for normalized hit rate:

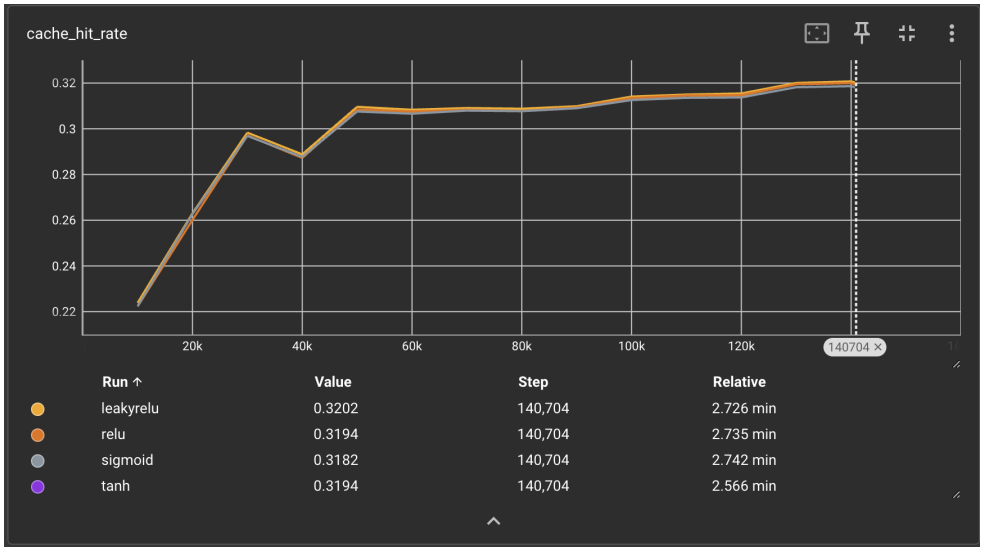


Task III

Vary the activation function of the multi-layer perceptron model and plot the normalized cache hit rate (y-axis) v/s the activation function (x-axis). Choose the following activation functions - ReLU, Leaky ReLU, sigmoid, tanh. Use the default code with appropriate modifications to the code. Change the activation functions after all the layers.

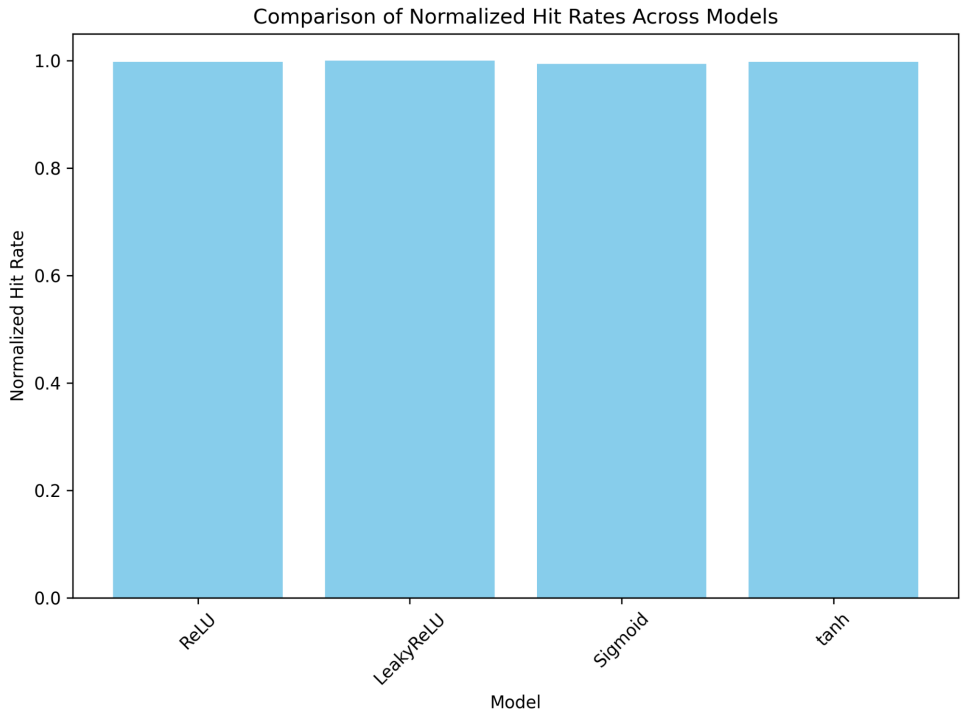
Raw hit rate:

From the chart, we can see that Leaky ReLU produces the highest final cache hit rate at around 0.3202, narrowly outperforming ReLU (0.3194) and tanh (0.3194). Meanwhile, sigmoid shows a slightly lower final performance of about 0.3182. Although the differences are not huge, these results suggest that Leaky ReLU can give a slight edge in this particular setup, likely because it avoids saturating gradients as aggressively as standard ReLU or sigmoid.



Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

Plot for normalized hit rate:

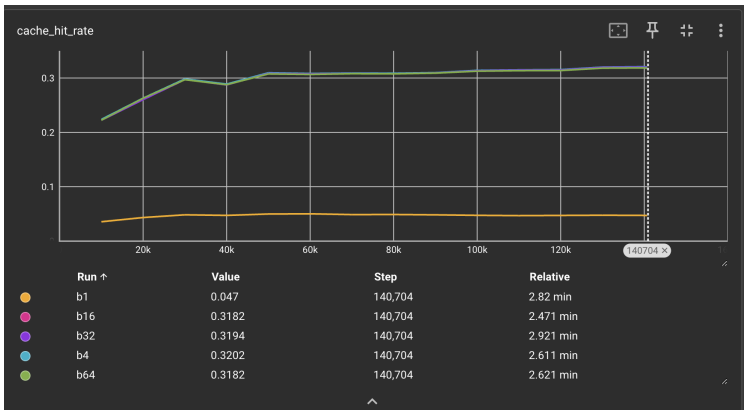


Task IV

Vary the batch size for the multi-layer perceptron model and plot the normalized cache hit rate (y-axis) v/s the batch size (x-axis) for batch sizes = 1(incremental), 4, 16, 32 (default), 64. This would be logged in the tensorboard directory and you may paste a screenshot of the tensorboard output. Use the default code, with appropriate modifications.

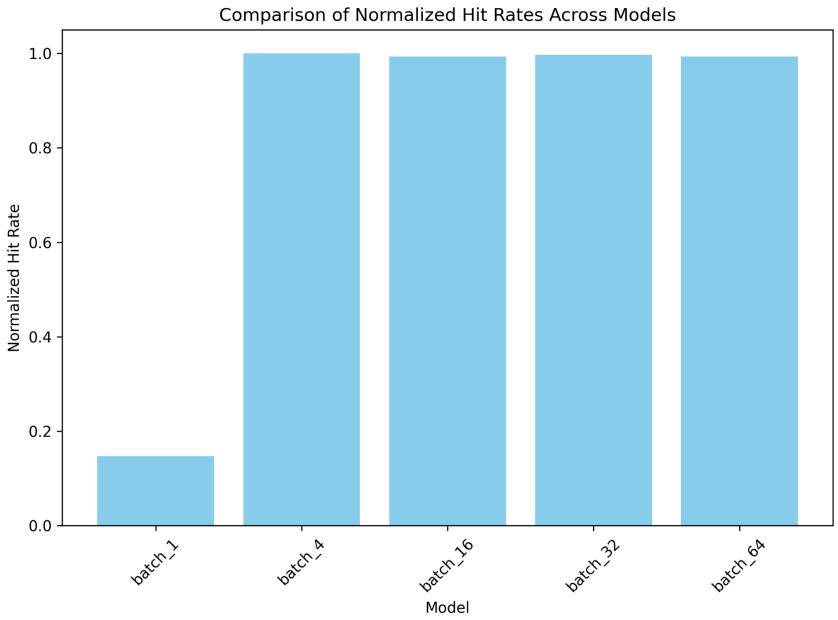
Raw hit rate for batch sizes = 1(incremental), 4, 16, 32 (default), 64:

The results show that extremely small batch sizes can lead to unstable or suboptimal learning, while moderately larger batches—such as 32—strike a good balance between convergence and performance. Going beyond 32 to 64 does not yield a dramatic boost, indicating that moderate batch sizes are likely the most efficient for this particular model and trace.



Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

Plot for normalized hit rate:

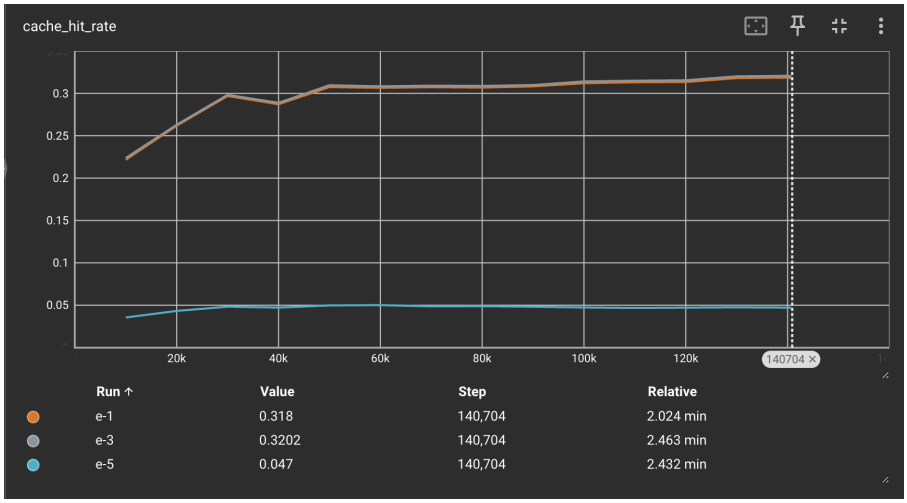


Task V

Vary the learning rate for the multi-layer perceptron model and plot the normalized cache hit rate (y-axis) v/s the number of epochs (x-axis) for learning rates = 0.00001, 0.001(default), 0.1. This would be logged in the tensorboard directory and you may paste a screenshot of the tensorboard output. Use the default code with appropriate modifications.

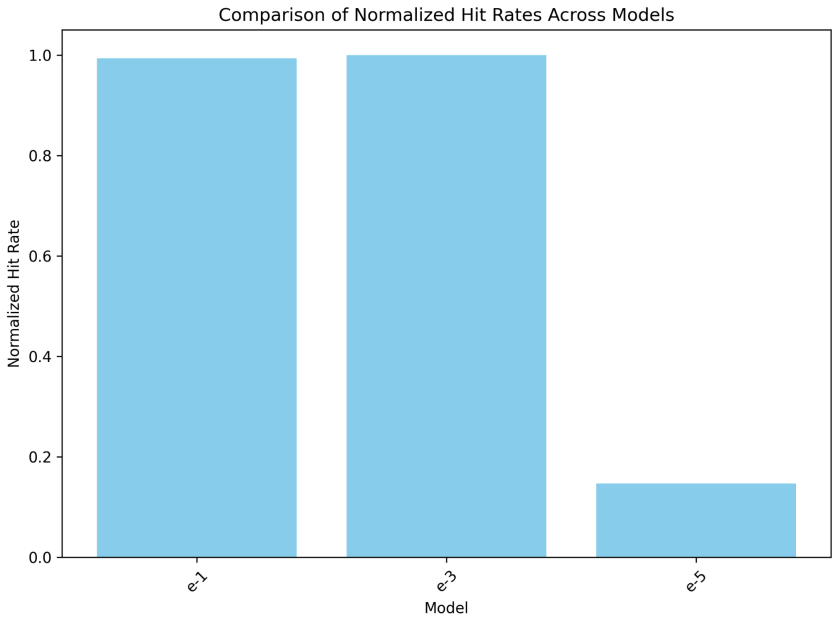
Raw hit rate for learning rates = 0.00001, 0.001(default), 0.1:

The result shows that the model converges poorly with a very small learning rate of 0.00001, ending up with a final cache hit rate of around 0.047—the model likely fails to converge effectively at such a low step size. At the other extreme, a very large learning rate of 0.1 converges to around 0.318, taking longer to reach its plateau. The default learning rate of 0.001 yields the best performance, with a final hit rate of about 0.3202, which is a “sweet spot” for this particular dataset and model configuration.



Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

Plot for normalized hit rate:



Task VI (Optional - Extra Credit)

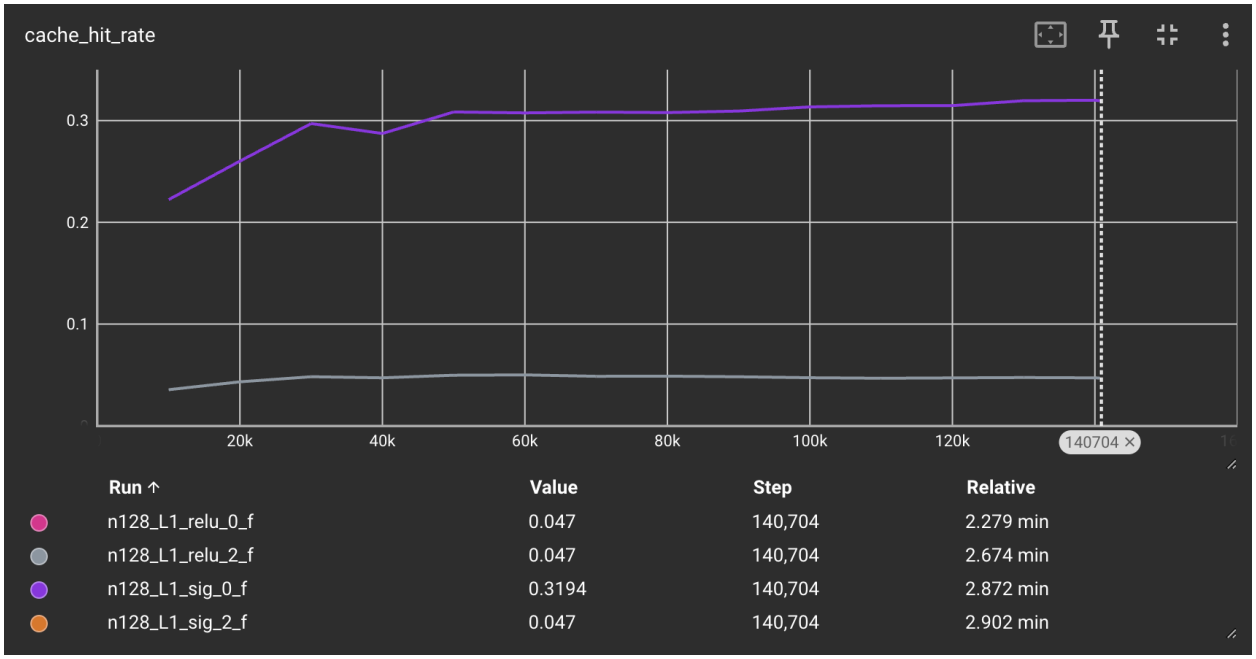
Do some more exploration, varying the number of neurons, number of layers, activation functions, dropout, pruning, etc to find the best normalized cache hit rate. Justify your observations.

In this task, I try various conditions:
neurons: [128]
layers: [1, 2]
activation: [ReLU, sigmoid]
dropout: [0.0, 0.2]
pruning: [false, true]

And use grid search for these 16 conditions to find the parameters to get the best normalized cache hit rate.
(Data name with form [neurons][layers][activation][dropout][pruning])

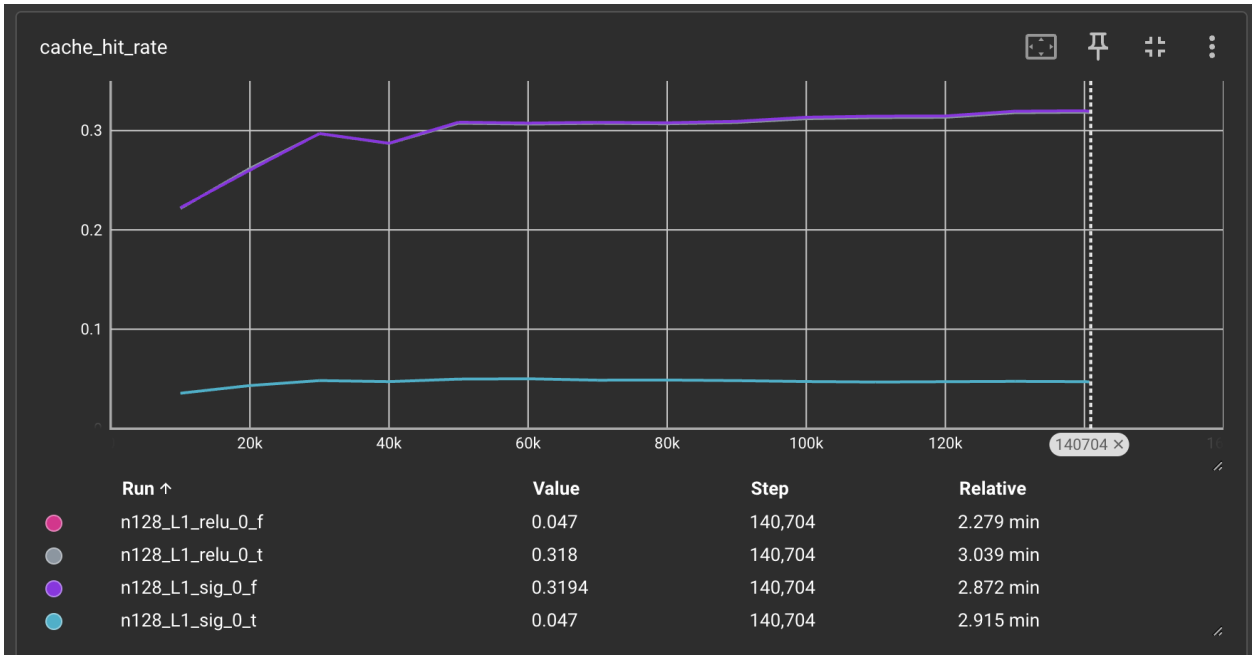
Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

For Layer 1, compare different activation and dropout rate:
We find that dropout rate will have a significant impact on the sigmoid activation, and sigmoid without dropout have a better performance in that case.



Run ↑	Value	Step	Relative
n128_L1_relu_0_f	0.047	140,704	2.279 min
n128_L1_relu_2_f	0.047	140,704	2.674 min
n128_L1_sig_0_f	0.3194	140,704	2.872 min
n128_L1_sig_2_f	0.047	140,704	2.902 min

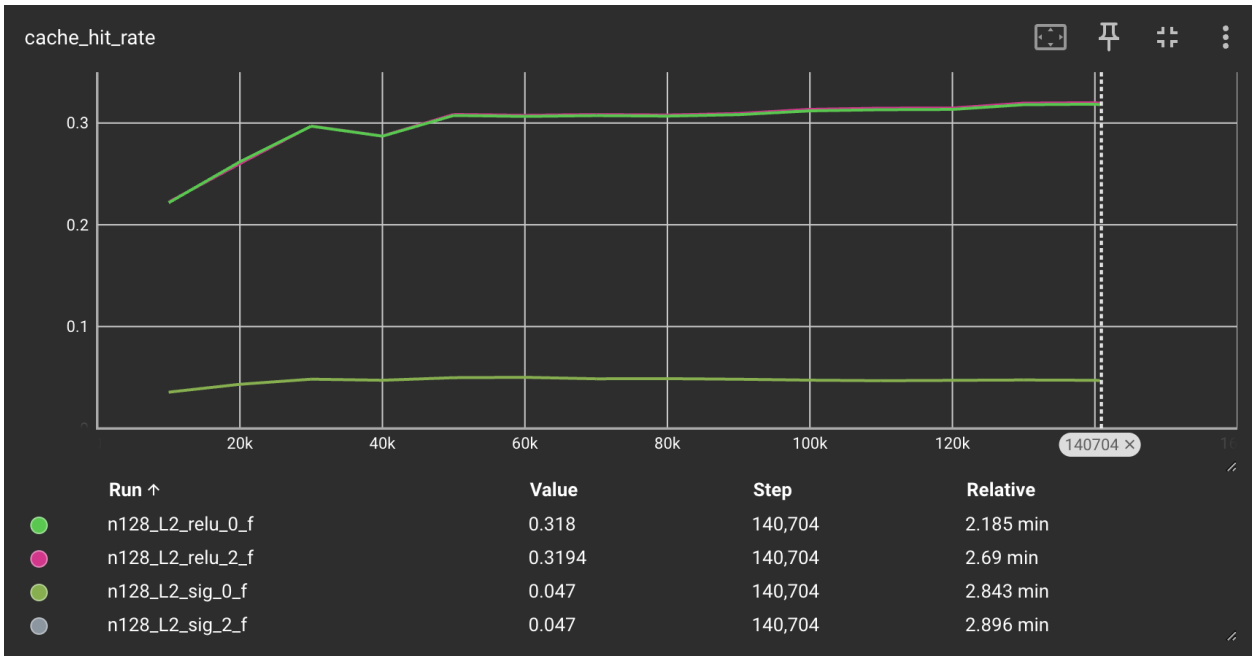
For Layer 1, compare different activation and pruning:
We find that ReLU with pruning have a better performance, while sigmoid perform better without pruning.



Run ↑	Value	Step	Relative
n128_L1_relu_0_f	0.047	140,704	2.279 min
n128_L1_relu_0_t	0.318	140,704	3.039 min
n128_L1_sig_0_f	0.3194	140,704	2.872 min
n128_L1_sig_0_t	0.047	140,704	2.915 min

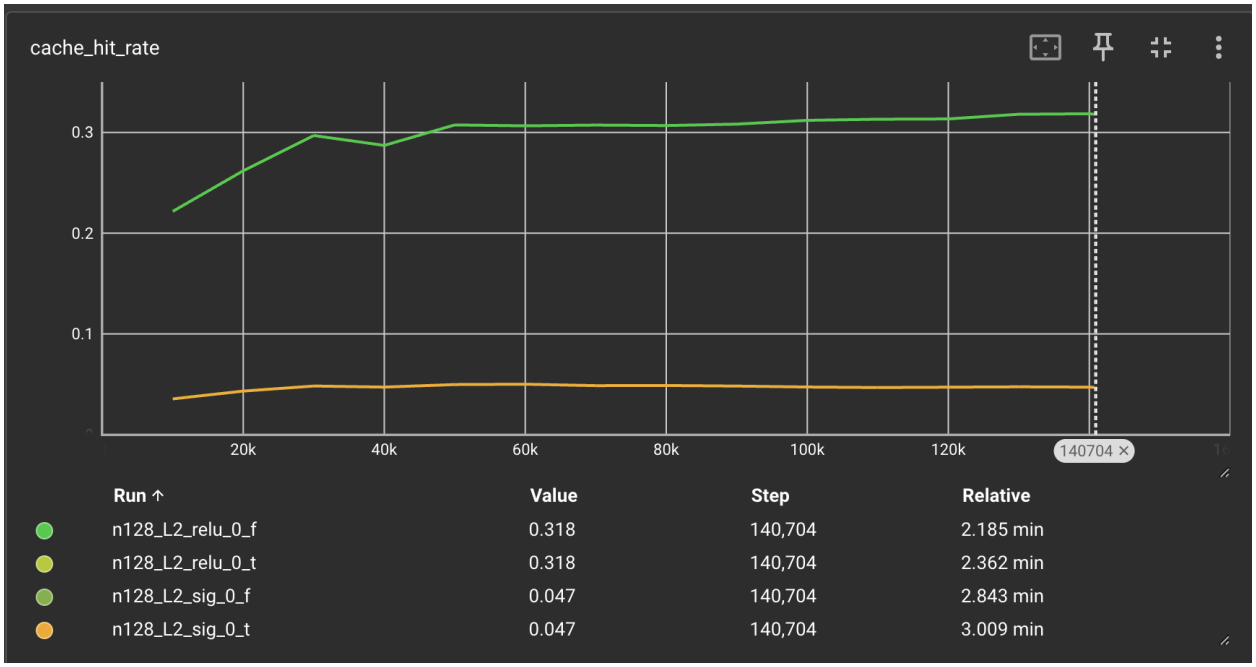
Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

For Layer 2, compare different activation and dropout rate:
We find that ReLU activation performs more stable in 2 layers compared to 1 layer for both with dropout or not dropout, and sigmoid function has low performance compared with single layer case.



Run ↑	Value	Step	Relative
n128_L2_relu_0_f	0.318	140,704	2.185 min
n128_L2_relu_2_f	0.3194	140,704	2.69 min
n128_L2_sig_0_f	0.047	140,704	2.843 min
n128_L2_sig_2_f	0.047	140,704	2.896 min

For Layer 2, compare different activation and pruning:
ReLU activation performs more stable in 2 layers compared to 1 layer for both with pruning or not, and sigmoid function still performs low in 2 layers case.



Run ↑	Value	Step	Relative
n128_L2_relu_0_f	0.318	140,704	2.185 min
n128_L2_relu_0_t	0.318	140,704	2.362 min
n128_L2_sig_0_f	0.047	140,704	2.843 min
n128_L2_sig_0_t	0.047	140,704	3.009 min

Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

In summary, we found that the best hit rate is around 0.3194,which is achieved either by (a) single-layer Sigmoid without pruning or (b) two-layer ReLU (with or without pruning). The exact choice might come down to resource considerations, since a single-layer Sigmoid is simpler, while a two-layer ReLU could be more robust in other scenarios.

Section IV

In this section you will experiment with a Recurrent Neural Network (RNN) model. You may find the code for this in “models/RNN_without_Attention” and“models/RNN_with_Attention” .

Deliverables [Section IV: Task 1-3]:

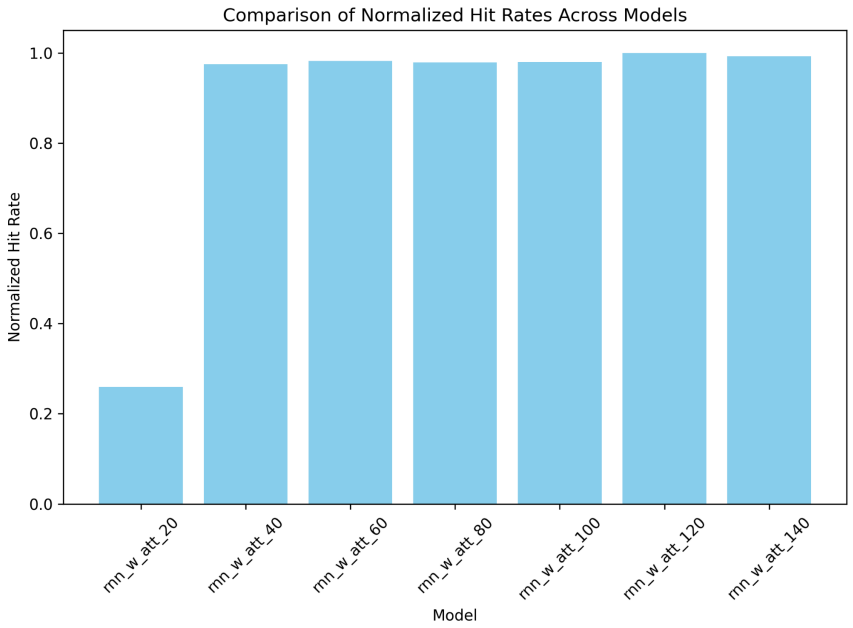
Task I

Vary the history length of the RNN with the Attention model and plot the normalized cache hit rate (y-axis) v/s the history length (x-axis). Plot the following history lengths - 20, 40, 60, 80, 100, 120, 140. You may change the “sequence_length” model_bindings argument in the command.

Tensorboard result of raw hit rate for different history lengths of RNN with the Attention model:
We found that an attention history length of 20 underperforms significantly (final hit rate ~0.0846), while larger history lengths of 40, 60, 80, 100, 120, and 140 yield final hit rates in the 0.31–0.33 range. Having at least 40 steps of history is beneficial, but the performance tends to plateau beyond 60–80 steps.

Run	Value	Step	Relative
rnn_w_att 100	0.3198	140,704	6.125 min
rnn_w_att 120	0.3264	140,704	3.367 min
rnn_w_att 140	0.3239	140,704	7.413 min
rnn_w_att 20	0.0846	140,704	4.732 min
rnn_w_att 40	0.3182	140,704	6.829 min
rnn_w_att 60	0.3205	140,704	3.351 min
rnn_w_att 80	0.3196	140,704	3.344 min

Plot of normalized cache hit rate:

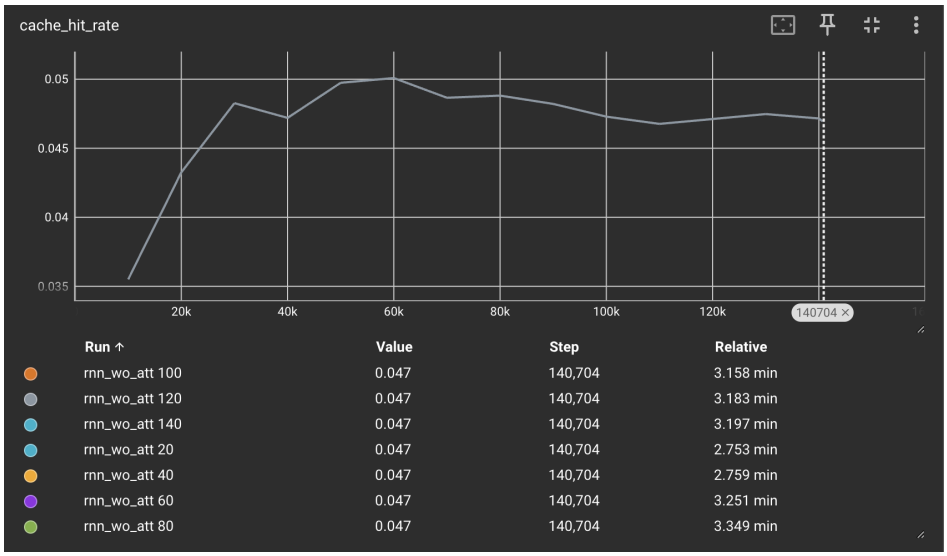


Task II

Vary the history length of the RNN without Attention model and plot the normalized cache hit rate comparing RNN with & without Attention (y-axis) v/s the history length (x-axis). Plot the following history lengths - 20,40,60,80,100,120,140. Use the data from the previous task.

Tensorboard result of raw hit rate:

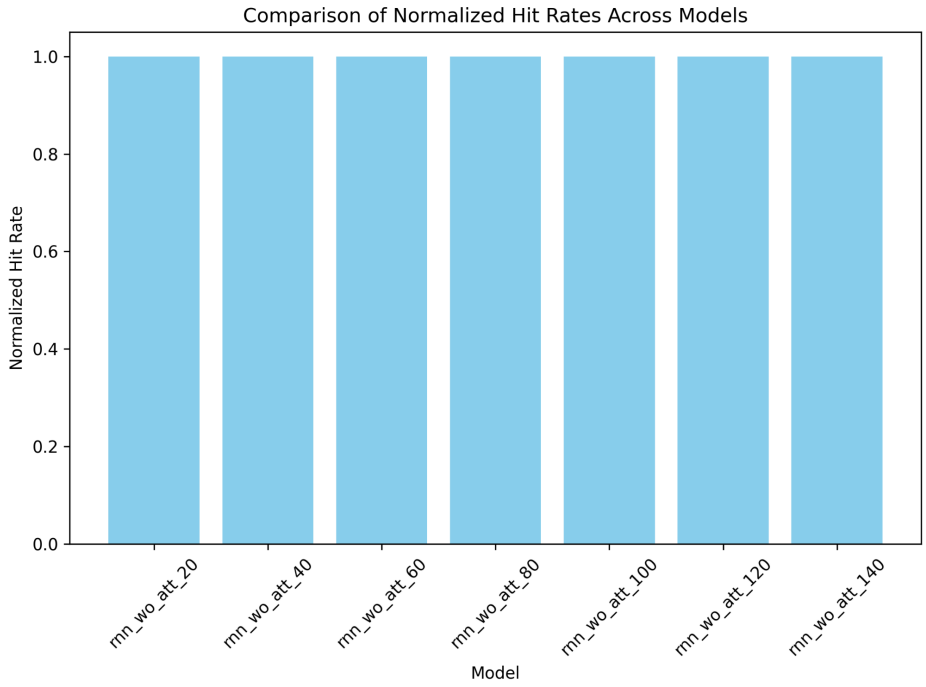
When using RNN without attention, all results perform low in that case, showing that only the RNN model doesn't catch the pattern well. Hence, incorporating Attention seems more beneficial than merely increasing the RNN's history length.



Run ↑	Value	Step	Relative
rnn_wo_att 100	0.047	140,704	3.158 min
rnn_wo_att 120	0.047	140,704	3.183 min
rnn_wo_att 140	0.047	140,704	3.197 min
rnn_wo_att 20	0.047	140,704	2.753 min
rnn_wo_att 40	0.047	140,704	2.759 min
rnn_wo_att 60	0.047	140,704	3.251 min
rnn_wo_att 80	0.047	140,704	3.349 min

Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

Plot of normalized cache hit rate:

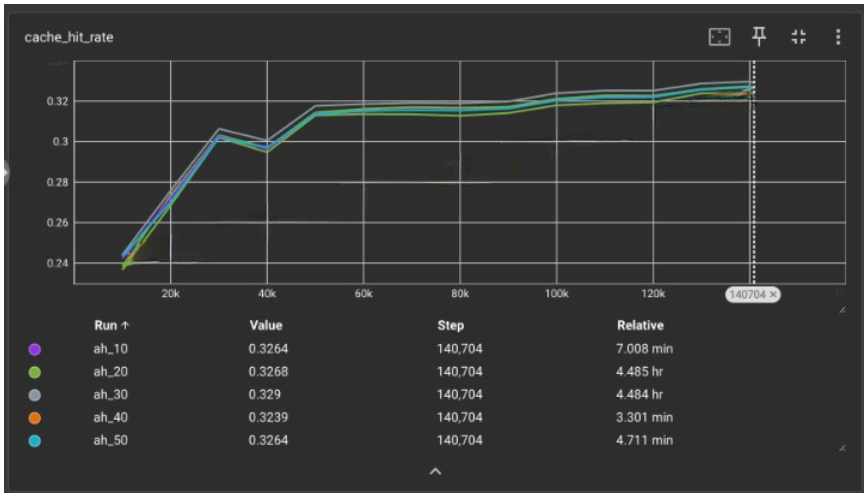


Task III

Vary the size of attention_history_length of the RNN with the Attention model and plot the normalized cache hit rate (y-axis) v/s the attention history length (x-axis). Plot the following attention history lengths - 10,20,30,40,50.

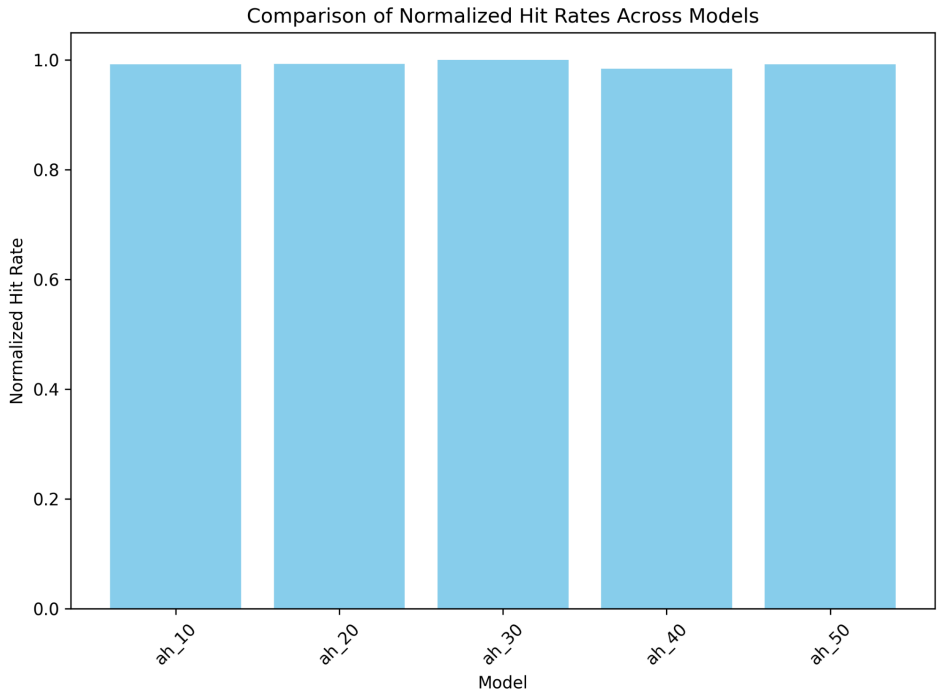
Tensorboard result of raw hit rate:

For RNN with different attention history length, runs with lengths 40 and 50 both reach final cache hit rates around 0.328, which is only slightly higher than shorter history lengths like 30. Although a larger attention history length can boost performance, it also increases computational cost and complexity. Thus, choosing 40 or 50 may not yield enough improvement over 30 to justify the added overhead



Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

Plot of normalized cache hit rate:



Section V

You will run all the models in this homework only on one trace file. You may find the astar_313B trace files (train, valid & test) in the "traces" directory. In this section you will perform ablation studies for the original LSTM model architecture. You may find the code for this in "models/LSTM". You can refer to the author's GitHub for some of the exact commands to perform ablation studies.

Deliverables [Section V: Task 1-4]:

Task I

Run the base configuration to have a baseline result for comparison. To do this run the following command:

```
Unset
python3 -m cache_replacement.policy_learning.cache_model.main \
  --experiment_base_dir=/tmp \
  --experiment_name=sample_model_llc \
```

```
--cache_configs=cache_replacement/policy_learning/cache/configs/default.
json \
  --model_bindings="loss=[\"ndcg\", \"reuse_dist\"]" \
  --model_bindings="address_embedder.max_vocab_size=5000" \

--train_memtrace=cache_replacement/policy_learning/cache/traces/sample_t
race.csv \

--valid_memtrace=cache_replacement/policy_learning/cache/traces/sample_t
race.csv
```

Task II

Run the following command and compare the result of changing the embedder by plotting it against the baseline:

python3 -m cache_replacement.policy_learning.cache_model.main \
 --experiment_base_dir=/tmp \
 --experiment_name=sample_model_llc \
 --cache_configs=cache_replacement/policy_learning/cache/configs/default.json \
 --model_bindings="loss=[\"ndcg\", \"reuse_dist\"]" \
 --model_configs=cache_replacement/policy_learning/cache_model/configs/default.json \
 --model_configs=cache_replacement/policy_learning/cache_model/configs/byte_embedder.json \
 --train_memtrace=cache_replacement/policy_learning/cache/traces/sample_trace.csv \
 --valid_memtrace=cache_replacement/policy_learning/cache/traces/sample_trace.csv

Task III

Run the following command and compare the result of ablating the reuse distance loss by plotting it against the baseline:

```
Unset
python3 -m cache_replacement.policy_learning.cache_model.main \
  --experiment_base_dir=/tmp \
  --experiment_name=sample_model_llc \

--cache_configs=cache_replacement/policy_learning/cache/configs/default.
json \
```

Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

```
  --model_bindings="loss=[\"ndcg\"]" \
  --model_bindings="address_embedder.max_vocab_size=5000" \

--train_memtrace=cache_replacement/policy_learning/cache/traces/sample_t
race.csv \

--valid_memtrace=cache_replacement/policy_learning/cache/traces/sample_t
race.csv
```

Task IV

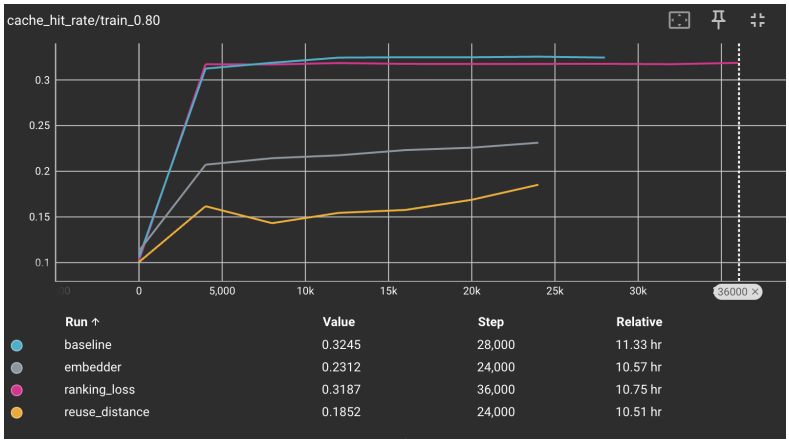
Run the following command and compare the result of ablating the ranking loss by plotting it against the baseline:

Unset

```
python3 -m cache_replacement.policy_learning.cache_model.main \
  --experiment_base_dir=/tmp \
  --experiment_name=sample_model_llc \

--cache_configs=cache_replacement/policy_learning/cache/configs/default.
json \
  --model_bindings="loss=[\"log_likelihood\", \"reuse_dist\"]" \
  --model_bindings="address_embedder.max_vocab_size=5000" \

--train_memtrace=cache_replacement/policy_learning/cache/traces/sample_t
race.csv \

--valid_memtrace=cache_replacement/policy_learning/cache/traces/sample_t
race.csv
```

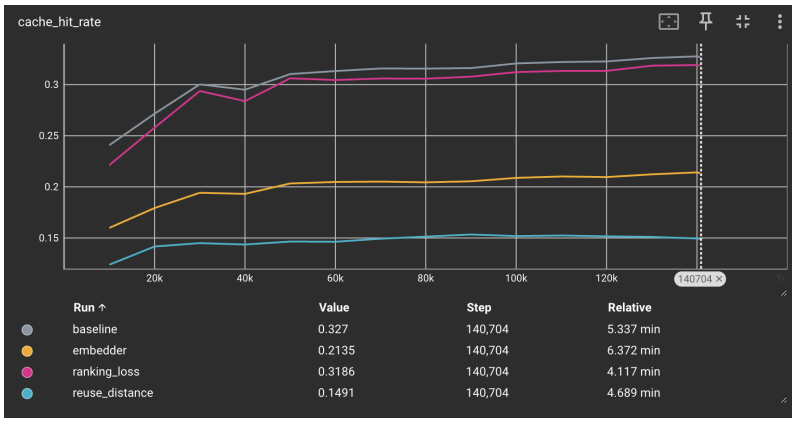
Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

train/valid result for Task 1 to Task 4:



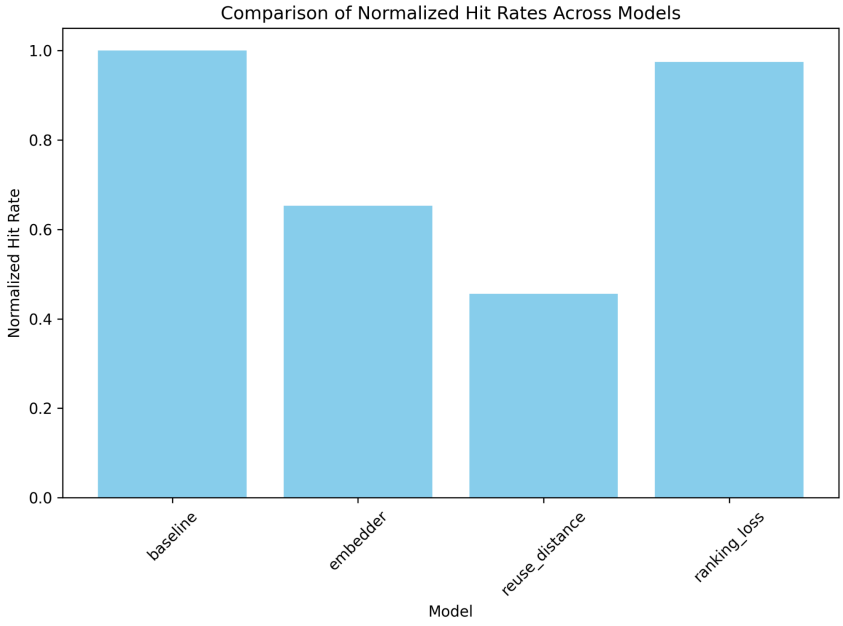
Run ↑	Value	Step	Relative
baseline	0.3245	28,000	11.33 hr
embedder	0.2312	24,000	10.57 hr
ranking_loss	0.3187	36,000	10.75 hr
reuse_distance	0.1852	24,000	10.51 hr

Test result for Task 1 to Task 4:



Run ↑	Value	Step	Relative
baseline	0.327	140,704	5.337 min
embedder	0.2135	140,704	6.372 min
ranking_loss	0.3186	140,704	4.117 min
reuse_distance	0.1491	140,704	4.689 min

Normalized hit rate:



Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)

From the final results, we see that the baseline approach yields the highest cache hit rate, hovering around 0.32 to 0.33. When we ablate the ranking loss, the performance drops slightly to about 0.3186, indicating that ranking contributes to performance but is not strictly critical. On the other hand, removing the reuse distance loss is highly detrimental, with the cache hit rate plummeting to about 0.1491. This suggests that reuse distance is especially important for guiding the model toward effective cache replacement decisions.

Additionally, changing the address embedder (byte embedder) does not seem to improve performance in this particular configuration, landing around 0.2135—well below the baseline. Overall, the results underscore the importance of careful design and tuning of the model's objectives, as well as how crucial the reuse distance metric is for learning a better replacement policy. Tuning hyperparameters, adjusting loss weights, or experimenting with different traces may further optimize performance.

Tzu Tang(Leslie) Liu (tliu33@ncsu.edu)